

CSE 5324-001 – Project Proposal

Agile Object-Oriented Software Engineering Project | Submission 1

Project Title	PawConnect - Animal Shelter & Adoption Management System
Team Members	Harsha Prasad - 1002230012 Himanshu Shimpi - 1002304702 Dipanjana Das - 1002327740
Class Number	CSE 5324-001
Semester	Fall 2026
Instructor	Farnaz Farahanipad
Project Title	PawConnect - Animal Shelter & Adoption Management System

1. Executive Summary and Product Vision

1.1 Product Vision

- PawConnect is a web-based Animal Shelter and Adoption Management System designed for a small animal shelter home that manages dogs and cats. The system provides a centralized platform where visitors can browse available animals. Additionally, registered adopters can submit adoption applications, schedule appointments, and monitor the status of their adoption process.
- A single authorized administrator (admin) will manage animal information, medical records, applications, appointments, animal status changes, and completed adoption records. The goal of PawConnect is to simplify the shelter's daily operations while providing adopters with a clear and organized adoption experience.

1.2 Problem / Opportunity Statement

- A registered adopter will be able to complete the primary workflow from viewing an animal through submitting and tracking an adoption application.
- The system will prevent invalid operations such as applying for an already adopted animal and creating conflicting appointments.
- The admin will be able to manage animal profiles, their medical records, applications, appointments, status changes, and completed adoption records from the one system.

1.3 Intended Users and Stakeholders

Stakeholder / User Type	Need / Goal	How the System Supports It
Visitor	Browse or explore animals available for adoption.	Browse, search, and view animal profiles without creating an account .
Registered Adopter	Apply for an animal.	Submit an adoption application after logging in.
Registered Adopter	Monitor adoption progress.	View application and appointment status.

Stakeholder / User Type	Need / Goal	How the System Supports It
Registered Adopter	Meet an animal.	Schedule an appointment after the application reaches the appropriate stage.
Administrator (Admin)	Maintain shelter animal information	Add and update animal profiles and their statuses
Administrator (Admin)	Maintain animal health information	Create and update medical records.
Administrator (Admin)	Review adoption requests	Approve or reject adoption applications request.
Administrator (Admin)	Manage adoption operations	Manage appointments, adoption completion, and adoption history.
Shelter	Maintain records	Adoption history and medical records

2. Project Scope and Objectives

2.1 In-Scope Features

- User registration, login, and logout.
- Adopter profile and contact information.
- Public browsing of dog and cat listings.
- Search and filtering by animal type, gender and status.
- Detailed animal profiles with name, photo, breed, age, gender, description, availability, and health information.
- Adoption application submission by registered adopters.
- Administrator (Admin) review of applications.
- Application approval and rejection.
- Appointment scheduling for adopters.
- Validation to prevent appointment conflicts.
- Validation to prevent adoption applications for animals that are already adopted.
- Administrator (Admin) management of animal records.
- Medical record management.
- Animal status management.
- Fostered animal status.
- Adoption completion.
- Adoption history.
- Role-based access for adopter and administrator functionality.

2.2 Out-of-Scope Items

The following items are intentionally excluded to maintain a realistic semester scope:

- Online payments.
- Donations.
- Veterinary billing.
- Multiple shelter branches.
- Multiple administrator levels.
- Native Android/iOS applications.
- AI-based pet recommendations.
- Live chat.
- External background-check services.

- External shelter integrations.
- Full foster-family registration/management.
- Transportation services.

Fostered will be modeled as an animal status, not as a separate foster-management subsystem.

2.3 Project Objectives / Success Criteria

- A registered adopter will be able to complete the primary workflow from viewing an animal through submitting and tracking an adoption application.
- The system will prevent invalid operations such as applying for an already adopted animal and creating conflicting appointments.
- The administrator will be able to manage animal profiles, medical records, applications, appointments, status changes, and completed adoption records from one system.

3. Team Organization and Collaboration Plan

Although each member has a primary responsibility, all three members will participate in requirements analysis, object-oriented design, UML modeling, database design, coding, testing, peer review, documentation, sprint reviews, and the final presentation.

Team Member	Primary Responsibilities
Harsha Prasad	Product Owner + Developer; requirements management, product backlog, authentication, adopter profile, adoption application module.
Himanshu Shimpi	Scrum Master + Developer; sprint coordination, animal listing/search, animal management, medical records.
Dipanjana Das	Technical Lead + Developer; appointments, admin dashboard, animal status workflow, adoption completion/history.

4. Agile Development Approach

The team will follow an Agile Scrum-style development process. Development will be iterative so that a small working increment is designed, implemented, tested, and demonstrated during each sprint.

4.1 Sprint / Iteration Structure

Item	Team Plan
Sprint length	2 weeks
Sprint planning	At the beginning of each sprint, review the prioritized Product Backlog, establish a Sprint Goal, select achievable user stories, and divide them into tasks .
Regular Scrum Meetings	Short coordination meetings 2-3 times per week to discuss completed work, upcoming work, and blockers.
Sprint review / demo	At the end of every sprint, demonstrate the completed working increment .
Retrospective	After every Sprint Review, discuss what worked, problems encountered, and improvements for the next sprint
Backlog tool	GitHub Projects
Version Control	Git and GitHub
Code Integration	Feature branches and pull requests

5. Initial Requirements / Evolving SRS Baseline

5.1 System Context and Assumptions

System context: PawConnect is a web-based application for small animal shelters. Visitors can browse dogs and cats; registered adopters may apply for adoption, request appointments, and track their activity; one authorized Administrator manages shelter records and workflows.

Assumptions: Only dogs and cats are supported. The shelter operates at one location. There is one predefined Administrator (Admin) account. Foster care is represented only as an animal status, not as a separate foster-family management subsystem.

Operational assumptions: Adopters must register and log in before submitting an adoption application or appointment request. The system stores adopter contact data and basic animal medical records. Appointments are shelter visit/meet requests and do not require an external calendar service.

5.2 Functional Requirements

	Functional Requirement	Priority	Acceptance / Verification Ide
FR-01	The system allows a new adopter to register using first name, last name, email, phone number, and password	Must	Register with valid data; verify a unique adopter account is stored. Duplicate email is rejected.
FR-02	The system shall allow registered adopters and the Administrator to log in and log out.	Must	Valid credentials start a session; invalid credentials are rejected; logout ends access to protected pages.
FR-03	The system shall allow a registered adopter to view and update basic profile/contact information.	Must	Update name or phone number and verify the saved values persist.
FR-04	The system shall allow visitors and adopters to browse dog and cat listings without requiring login.	Must	Open the listing when logged out and verify animal cards are displayed.
FR-05	The system shall allow users to search/filter animals by type, breed, age, and current status.	Must	Apply each filter and verify only matching animals are displayed; clearing filters restores the list.
FR-06	The system shall display a detailed animal profile including name, photo, type, breed, age, gender, description, status, and basic health information.	Must	Open an animal and verify all required profile fields are shown.
FR-07	The Administrator shall be able to add a new dog or cat record.	Must	Create an animal with valid data and verify it appears in the listing.
FR-08	The Administrator shall be able to update animal profile information and current status.	Must	Edit an animal, save changes, and verify the updated values are displayed.
FR-09	The Administrator shall be able to create, view, and update medical records for an animal.	Must	Add a vaccination/treatment record and verify it is linked to the correct animal.
FR-10	A logged-in adopter shall be able to submit an adoption application for an eligible animal.	Must	Submit valid applications and verify a pending application is created for the adopter and animal.
FR-11	The system shall prevent an adoption application from being submitted for an animal that is already adopted.	Must	Attempt to apply for an Adopted animal and verify the request is blocked with a clear message.
FR-12	A registered adopter shall be able to view the status of their own adoption applications.	Must	Log in as an adopter and verify only that adopter's application statuses are visible.
FR-13	The Administrator shall be able to view all submitted adoption applications.	Must	Admin opens application management and sees pending/submitted applications with adopter and animal details.
FR-14	The Administrator shall be able to approve or reject an adoption application.	Must	Approve/reject a Pending application and verify the decision is saved and visible to the adopter.

FR-15	The system shall maintain animal states including Available, Application Pending, Approved, Fostered, and Adopted.	Must	Trigger valid workflow actions and verify the animal status changes to the expected state.
FR-16	The system shall allow the Administrator to view the previous adoption record of an adopted animal.	Must	Select an animal with status Adopted and verify that the Administrator can view the associated adoption record, including adopter name, animal name, adoption date, and related adoption/application information
FR-17	A registered adopter shall be able to request an appointment for an eligible animal by selecting a date and time.	Must	Submit a valid appointment request and verify it appears with Requested status.
FR-18	The system shall prevent conflicting appointments for the same animal/date/time slot.	Must	Create an appointment, then attempt the same animal/date/time and verify the duplicate conflict is rejected.
FR-19	The Administrator shall be able to view and update appointment requests.	Must	Admin changes a request to Confirmed, Completed, or Cancelled and the adopter sees the updated status.
FR-20	The Administrator shall be able to complete an approved adoption.	Must	Complete an approved adoption and verify the animal becomes Adopted and a completed adoption record is created.
FR-21	The system shall maintain adoption history and allow the Administrator to review completed adoption records.	Must	Complete an adoption and verify the adopter, animal, and adoption date appear in history.
FR-22	The system shall enforce role-based access so adopter functions and Administrator management functions are separated.	Must	Attempt Admin routes as a visitor/adopter and verify access is blocked; verify the Administrator can access them.

5.3 Nonfunctional Requirements

	Quality Attribute	Requirement / Target
1	Usability	A visitor should be able to get from the homepage to an animal's profile in three clicks or fewer, and every form needs to show a clear message when something's filled in wrong.
2	Performance	For the academic dataset, common pages and search/filter requests should return within 2 seconds under normal local/demo use.
3	Security / Privacy	Passwords are never stored as plain text, only as hashes. Any page showing personal or admin data has to check the user is logged in with the right role first, and adopters should only ever see their own applications and appointments, not anyone else's.
4	Reliability	The system shall validate required fields and invalid state transitions and shall not create duplicate/conflicting appointment or adoption records.
5	Compatibility	The app should work properly on current versions of Chrome and Edge, and stay usable at normal laptop and desktop screen sizes.

5.4 Constraints and External Interfaces

- **Technology constraints:** The product will be implemented as a web application using the team-selected frontend, backend, database, Git/GitHub, and GitHub Projects. Development will use feature branches and pull requests.
- **Database / storage constraints:** The baseline design uses core entities for User, Animal, AdoptionApplication, Appointment, MedicalRecord, and Adoption, with animal status-change history stored either as a small supporting entity/table or audit records.
- **Third-party APIs or services:** No external payment, map, SMS, email-verification, calendar, background-check, or shelter-integration API is required for the baseline release.
- **Device/browser/platform constraints:** Desktop/laptop web browsers are the primary target; Chrome and Edge will be used for acceptance testing. A native Android/iOS application is out of scope.

- Legal/privacy/safety constraints, if applicable: Store only project-relevant adopter contact information, never store plaintext passwords, restrict administrative data/functions to the authorized Administrator, and avoid presenting medical notes as professional veterinary advice.

6. Product Backlog and User Stories

Story ID	User Story	Priority	Planned Sprint
US-01	As a visitor, I want to register and log in as an adopter so that I can submit adoption applications and appointment requests.	High	Sprint 1
US-02	As a visitor/adopter, I want to browse dogs and cats and search/filter them so that I can find animals that match my preferences.	High	Sprint 1
US-03	As a visitor/adopter, I want to view a detailed animal profile so that I can understand the animal before applying or requesting a visit.	High	Sprint 1
US-04	As an Administrator, I want to add and update animal profiles and medical records so that shelter information stays accurate.	High	Sprint 2
US-05	As a registered adopter, I want to submit and track an adoption application so that I know the progress of my request.	High	Sprint 2
US-06	As an Administrator, I want to review adoption applications and approve or reject them so that adoption requests are controlled by the shelter.	High	Sprint 2
US-07	As an adopter and Administrator, I want appointment scheduling and adoption completion/history so that meetings and completed adoptions are recorded correctly.	High	Sprint 3

6.1 Acceptance Criteria for Highest-Priority Stories

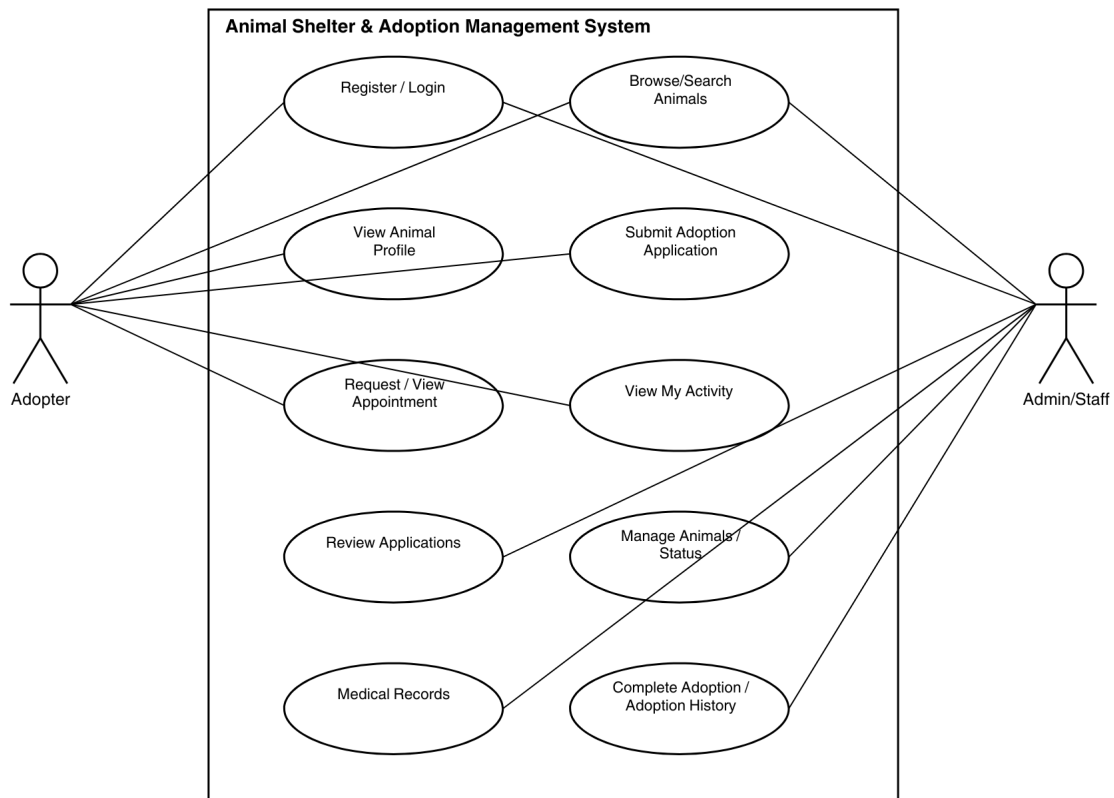
- US-01 acceptance criteria: Given a visitor provides valid unique registration data, when the form is submitted, then an adopter account is created; given valid credentials, when login is submitted, then the adopter is authenticated; invalid credentials or duplicate email produce a clear error.
- US-02 acceptance criteria: Given animal records exist, when the listing opens, then dogs and cats are displayed from the database; when type, breed, age, or status filters are applied, then only matching animals are shown; clearing filters restores the full list.
- US-03 acceptance criteria: Given an animal is selected, when its profile opens, then the required profile and health fields are displayed; if the animal is eligible, the adopter can proceed to the appropriate adoption/appointment action, while invalid actions for an Adopted animal are unavailable or blocked.

7. Initial Object-Oriented Analysis and UML

7.1 Use Case Model

- Initial problem-domain classes/objects: User (Visitor/Registered Adopter/Administrator role context), Animal, AdoptionApplication, Appointment, MedicalRecord, Adoption, and AnimalStatusHistory. Key relationships: one registered User can submit many AdoptionApplications and Appointments; one Animal can have many applications, appointments, medical records, and status-history records; a completed Adoption links one adopter, one animal, and the approved application.

UML Use Case Diagram - Animal Shelter & Adoption Management System



Scope note: animal types are limited to Dog and Cat; there is one predefined Admin/Staff account.

1. Primary actors: Visitor, Registered Adopter, and Administrator (one authorized shelter staff account).
2. Important use cases:
 - UC-01 Browse/Search/Filter Animals: Visitor or Registered Adopter browses dog/cat listings, applies filters, and opens a detailed animal profile.
 - UC-02 Submit and Track Adoption Application: A logged-in adopter applies for an eligible animal; the system creates the application and lets the adopter monitor its status.
 - UC-03 Review Adoption Application: The Administrator reviews the application and approves or rejects it; related animal/application states are updated.
 - UC-04 Request and Manage Appointment: A registered adopter requests a visit date/time; the system checks conflicts; the Administrator confirms, completes, or cancels the request.
 - UC-05 Complete Adoption and Maintain History: For an approved application, the Administrator records completion; the animal becomes Adopted and the system stores adoption/history information.

8. UI / UX Concept and Technical Feasibility

8.1 Initial Wireframes / Screen Flow (A wireframe is a very simple sketch of a screen. It does not need colors, final fonts, logos, or polished graphics.)

- Insert 3-6 low-fidelity wireframes or screen sketches.
- Identify the main navigation path for a primary user. (For example: Login→Home→Select Restaurant→Choose Item→Cart→Checkout)

8.2 Proposed Technology Stack

Layer / Need	Proposed Technology	Reason for Choice
Frontend	React.js + Bootstrap + CSS	React provides a simple component-based structure for pages such as Home, Animal Listing, Animal Profile, Adoption Application, Adopter Dashboard, and Admin Dashboard. Bootstrap/CSS will help create a responsive UI quickly.
Backend	Node.js + Express.js	Express provides a simple REST API for registration/login, animals, adoption applications, appointments, medical records, and adoption history. It also integrates easily with React and SQL.
Database	SQL	The project has structured relational data such as users, animals, adoption applications, appointments, medical records, and adoptions. The SQL database will be used to store and manage these relationships.
Hosting / Runtime	Node.js Runtime + Vercel	Node.js will be used as the application runtime for the backend, while Vercel will be used to host and deploy the web application for easy access and demonstration.
Version Control	Git / GitHub	Used for team collaboration, feature branches, commits, pull requests, version history, and integration through the PawConnect team repository.

9. Sprint Planning

Sprint 1 Story / Task	Owner(s)	Acceptance Criteria	Expected Evidence
Repo & project setup	ALL	Team can clone and run the app locally	GitHub repo, README, commits
Backlog board setup	ALL	All 7 stories loaded with priority/sprint labels	Board screenshot
US-01: Adopter registration	Harsha (UI), Himanshu (backend)	Account created; duplicate email blocked	Screenshots, commit
Password hashing	Himanshu	Password stored as a hash, not plain text	Code snippet/commit
US-02: Login/logout	Harsha (UI), Himanshu (auth)	Correct login works, wrong login blocked, logout ends session	Screen recording, commit
FR-03: Adopter profile view/update	Harsha	Adopter edits name/phone and it saves	Screenshot, commit
Animal data model & schema	Himanshu (Dipanjana reviews)	Schema stores/retrieves a test record	Schema file, DB screenshot
Seed data	Himanshu / Dipanjana	10–15 sample animals in the DB	Seed script, DB screenshot
US-03: Animal listing (cards)	Himanshu (API), Harsha (UI)	Animals load as cards from the DB	Screenshot, commit
FR-06: Animal detail profile page	Harsha	Selecting an animal shows full profile with health info	Screenshot, commit
Base admin dashboard	Dipanjana	Only the admin role can reach it	Screenshot (blocked + success), commit

Sprint 2 Story / Task	Owner(s)	Acceptance Criteria	Expected Evidence
US-04: Admin animal & medical record management	Himanshu (logic), Dipanjana (dashboard UI)	Add/edit animal and medical record saves and displays	Screenshots, commit
US-05: Submit & track adoption application	Harsha	Application created as Pending; blocked for already-Adopted animals	Screenshots, commit
US-06: Admin review (approve/reject)	Dipanjana (dashboard), Himanshu (status logic)	Decision updates the application and the animal's status	Screenshot, commit
Integration & testing	ALL	Browse → apply → admin review flow works end to end	Sprint demo, bug list
Sprint review & retro	ALL	Team demos the flow, notes fixes for Sprint 3	Meeting notes

Sprint 3 Story / Task	Owner(s)	Acceptance Criteria	Expected Evidence
FR-17/18: Appointment request + conflict check	Dipanjana	Valid request succeeds; same animal/date/time twice is rejected	Screenshots, commit
FR-19: Admin manages appointment status	Dipanjana	Confirm/complete/cancel is visible to the adopter	Screenshot, commit
FR-20: Admin completes an adoption	Dipanjana	Animal becomes Adopted; adoption record created	Screenshot + DB record, commit
FR-16/21: View completed adoption record	Dipanjana, Himanshu (backend)	Adopted animal shows adopter, animal, and adoption date	Screenshot, commit
FR-22: Role-based access check	Himanshu	Visitor/adopter blocked from admin routes; admin can access them	Screenshot, commit
Integration & testing	ALL	Full flow (browse → apply → approve → appointment → complete → history) runs end to end	Sprint demo, bug list
Sprint review & retro	ALL	Team demos final flow, preps for testing/final phase	Meeting notes

10. Risks, Dependencies, and Mitigation

Risk / Dependency	Likelihood	Impact	Mitigation / Contingency
Team hasn't finalized the frontend/backend/database stack yet, and members have uneven experience with whatever gets picked	Med	Med	Lock in the stack by the end of Week 1, favor tools at least one member already knows, and do a short practice build before Sprint 1 planning starts
Team scheduling conflicts - three members with different class schedules and outside commitments make it hard to find overlapping time	Med	Med	Set one fixed weekly sync meeting early in the semester; use the GitHub Projects board for async status updates between syncs
Scope is fairly large for one semester - 22 functional requirements is a lot to fully implement and test with three people	Med	High	Treat "Must" priority items (core browse → apply → review → adopt workflow) as the real deliverable; hold anything else as stretch and re-check the backlog after every sprint review
Merge conflicts and integration issues	Med	Med	Use small feature branches, commit often in small

from three people working in the same codebase at once			chunks, and require a pull request review before merging into main instead of large infrequent merges
--	--	--	---

11. COCOMO-Based Project Effort and Cost Estimation

11.1 Estimate Software Size (KLOC)

COCOMO Input	Team Estimate / Selection	Assumptions / Justification	Evidence / Basis
Estimated source size (LOC)	6000 LOC	Estimated by breaking the system into the functional groups already defined in our FR table (Section 5.2) and assigning a rough LOC figure to each group	See component breakdown below
Estimated source size (KLOC)	6 KLOC	$6,000 \div 1,000$	Direct conversion
Assumed labor cost (optional)	\$10,000 per person-month	Standard academic placeholder figure used for illustration, not an actual salary or contract rate	Academic estimate only

Component breakdown used to reach the 6,000 LOC figure, mapped directly to our functional requirements:

- Authentication and adopter account management (FR-01-FR-03): ~600 LOC
- Public browsing, search/filter, detailed animal profiles (FR-04-FR-06): ~800 LOC
- Admin animal and medical record management (FR-07-FR-09): ~700 LOC
- Adoption application workflow, submit, validate, view, review, approve/reject (FR-10-FR-14): ~900 LOC
- Animal status management and status-change history (FR-15-FR-16): ~500 LOC
- Appointment scheduling, conflict validation, admin appointment management (FR-17-FR-19): ~700 LOC
- Adoption completion and adoption history (FR-20-FR-21): ~500 LOC
- Role-based access control (FR-22): ~300 LOC
- Shared frontend layout/navigation, shared backend utilities, database configuration: ~1,000 LOC

11.2 Select and Justify the COCOMO Project Mode

We selected Organic mode. The team is a group of 3, the system is small in scope, and the problem domain, a CRUD-style web application with authentication, listings, applications, and an admin workflow, is a well-understood pattern rather than something novel or tightly constrained. There are no real-time, embedded, or unusually high-reliability requirements involved, which is what would push this toward Semi-detached or Embedded. Organic is the mode Basic COCOMO recommends for exactly this kind of small, familiar, low-constraint project.

Mode	Typical Fit	Effort Equation (E)	Schedule Equation (TDEV)
Organic	Small, relatively familiar projects with few rigid constraints	$E = 2.4 \times (\text{KLOC})^{1.05}$	$\text{TDEV} = 2.5 \times (E)^{0.38}$
Semi-detached	Intermediate size/complexity or mixed team experience	$E = 3.0 \times (\text{KLOC})^{1.12}$	$\text{TDEV} = 2.5 \times (E)^{0.35}$
Embedded	Tightly constrained or highly complex systems	$E = 3.6 \times (\text{KLOC})^{1.20}$	$\text{TDEV} = 2.5 \times (E)^{0.32}$

11.3 Perform the Basic COCOMO Calculation

Mode and equations used: Organic - $E = 2.4 \times (\text{KLOC})^{1.05}$, $\text{TDEV} = 2.5 \times (E)^{0.38}$

KLOC used: 6

Effort calculation:

$$E = 2.4 \times (6)^{1.05}$$

$$6^{1.05} \approx 6.56$$

$$E = 2.4 \times 6.56 \approx 15.8 \text{ person-months}$$

Development time and staffing:

$$\text{TDEV} = 2.5 \times (15.8)^{0.38}$$

$$15.8^{0.38} \approx 2.85$$

$$\text{TDEV} = 2.5 \times 2.85 \approx 7.1 \text{ months}$$

$$\text{Average staffing} = E \div \text{TDEV} = 15.8 \div 7.1 \approx 2.2 \text{ people working concurrently on average}$$

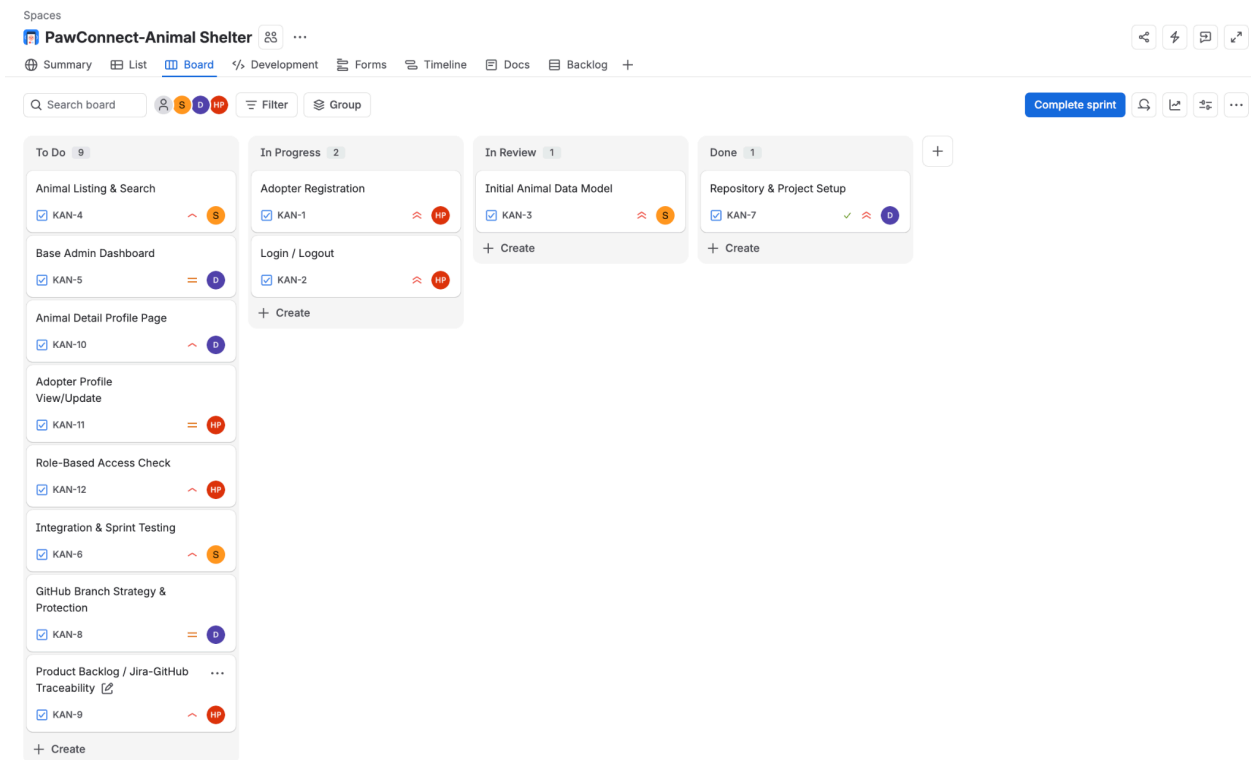
$$\text{Theoretical labor cost: } 15.8 \text{ person-months} \times \$10,000/\text{person-month} \approx \$158,000 \text{ (academic estimate only)}$$

12. Preliminary Semester Roadmap

Period	Planned Focus	Expected Deliverable
Weeks 1-4	Team formation, project topic selection, requirements elicitation, initial scope and backlog, first pass at the SRS baseline and UML use cases, while covering Chapters 1-4 in lecture	Submission 1: Proposal
Weeks 5-8	Domain modeling and use-case refinement, object interaction/sequence modeling, design class diagram, and UI design, matched with implementing Sprints 1 and 2 (registration/login, animal listings, animal/medical record management, the core adoption application workflow)	Submission 2: Progress Report
Weeks 9-End	Software QA, testing, maintenance and security, and project cost estimation topics, matched with completing Sprint 3 (appointment scheduling, adoption completion, adoption history), full testing pass, finalized UML/documentation, and presentation prep	Submission 3: Final Report + Product

13. Repository, Project Management, and Evidence Links

- Git repository URL: <https://github.com/hbprasad0110-rgb/PawConnect>
- Project board / backlog URL: [Backlog URL](#)



- Prototype / deployed URL, if available:

<https://www.figma.com/make/fPBMyMuMe3Cylmle2XkYO4/PawConnect-Web-App-Prototype?code-node-id=0-6&p=f&fullscreen=1>

14. References

1. Kung, D. C. *Software Engineering*, 2nd Edition, McGraw Hill - course reference for software engineering processes, requirements, UML, testing, and project management.
2. CSE 5324-001 Course Materials, Fall 2026 - course notes and project guidance.
3. React Documentation - frontend library reference.
4. Node.js and Express.js Documentation - backend runtime and REST API development reference.
5. Bootstrap Documentation - responsive UI and styling reference.
6. SQL Documentation - database design and relational data management reference.
7. Git and GitHub Documentation - version control, branching, pull requests, and team collaboration reference.
8. Vercel Documentation - deployment and hosting reference.
9. PawConnect GitHub Repository - project source code and team development repository.